

# Polaris in Mathematical Form

A companion to the Version 3 technical report

Draft of 23 September 2026

**Abstract**

This document restates Polaris, pre-pilot software, as a set of definitions and properties. It is a translation, not a proof: each property carries a mark saying how the repository supports it, and a property marked [enforced] is enforced by a named object in the schema or the code, not proved. Polaris is pre-pilot software on notional data. The only certification is of one component: `polaris-oid4vp 1.0.0rc7` is OpenID Certified by Egor Khaklin to the OpenID4VP 1.0 + HAIP 1.0 Verifier profile (24 September 2026). Nothing below claims more.

## 0. How to read the marks

| Mark | Meaning |
|---|---|
| [enforced] | a schema object, trigger, index, check constraint or code path makes it hold, and a check pins that object |
| [tested] | a test or drill exercises it; nothing forbids a violation outside the tested inputs |
| [measured] | a number observed on stated hardware at a stated version |
| [modelled] | a TLA$^+$ model checks it; that is evidence about the model |
| [assumed] | a cryptographic or operational assumption Polaris relies on and does not establish |
| [limit] | a stated bound on what Polaris does *not* provide |

## 0.1 A model, not the system

The equations below are a projection of Polaris, not Polaris. Write $\mathcal{R}$ for the system as it exists in the world (the code, the institutions that would run it, the people it would describe), $\mathcal{O}$ for what can be observed or evidenced about it, and $\mathcal{M}$ for this formal model. The relationship this document assumes is

$$\mathcal{R} \xrightarrow{\Omega} \mathcal{O} \xrightarrow{\Pi} \mathcal{M}, \qquad \Pi(X) \neq X, \qquad \mathcal{M} \not\equiv \mathcal{R}.$$

$\Omega$ is observation: tests, drills, measurements, and what outside parties have done. $\Pi$ is formalisation, and it loses information. What it keeps is what mathematics states well: admissible states, transition rules, invariants, verification equations, trust relations, bounds on what flows where. What it cannot carry is exactly what the project exists for: whether a person can be compelled, whether an institution is legitimate, whether an authority is independent of the one it federates with, whether a population consents. Assigning such a thing a variable does not capture it. The marks in Section 0 exist because of this gap: a property of $\mathcal{M}$ is a claim about $\mathcal{R}$ only as far as the evidence in $\mathcal{O}$ carries it.

Several distinctions the report insists on are instances of the same inequality, and the model keeps each of them separate rather than collapsing them:

$$\begin{aligned} \text{claim} &\neq \text{evidence},\\ \text{internal evidence} &\neq \text{external validation},\\ \text{signature validity } A(t) &\neq \text{current authority } R(t,\tau),\\ \text{a passing test} &\neq \text{the property it names},\\ \text{privacy} &\neq \text{freedom},\\ \text{protocol independence} &\neq \text{institutional independence}. \end{aligned}$$

What is true of Polaris in the world can exceed what this model establishes, and what the model establishes can exceed what has been observed. Reading an equation here as a guarantee about people would repeat, in mathematics, the error the report's corrections tables record in code.

# 1 The system

**Definition 1** (System)**.** $\mathfrak{P} = \langle \mathcal{P}, \mathcal{T}, \mathcal{A}, \mathcal{C}, \mathcal{E}, \mathcal{G}, \mathcal{K}, \mathcal{L}, \mathcal{W}, \mathcal{F} \rangle$*: persons, tokens (credentials), issuing authorities, the seven verification contexts, verification events, the table of cryptographic algorithms, keys, the transparency log, its witnesses, and the federation graph.*

**Definition 2** (Token)**.** *A token is* $t = (\nu_t, p_t, a_t, \alpha_t, s_t, x_t, prev_t)$*: its value* $\nu_t$ *(unique:* $\nu_t = \nu_{t'} \Rightarrow t = t'$*), person* $p_t \in \mathcal{P}$*, issuing authority* $a_t \in \mathcal{A}$*, algorithm* $\alpha_t \in \mathcal{G}$*, status* $s_t$*, expiry date* $x_t$ *(possibly* $\perp$*), and the token it succeeds, if any, with* $prev_t \neq t$ *and* $p_{prev_t} = p_t$*.*

# 2 The token state machine

Let $S = \{\textsc{reserve}, \textsc{active}, \textsc{dormant}, \textsc{lost}, \textsc{expired}, \textsc{revoked}\}$ and

$$\begin{aligned} \Gamma = \{&(\textsc{reserve}, \textsc{active}), (\textsc{reserve}, \textsc{revoked}), (\textsc{active}, \textsc{dormant}),\\ &(\textsc{active}, \textsc{revoked}), (\textsc{active}, \textsc{lost}), (\textsc{active}, \textsc{expired})\}. \end{aligned}$$

**Property 1** (Legal transitions)**.** *Every change of status is a pair in* $\Gamma$*: if* $s_t(\tau) \neq s_t(\tau')$ *for consecutive writes then* $(s_t(\tau), s_t(\tau')) \in \Gamma$*. Writes that do not change the status are unconstrained by* $\Gamma$*. And no write enters* ACTIVE *past the expiry: if* $s_t(\tau') = \textsc{active} \neq s_t(\tau)$ *then* $x_t = \perp \vee \tau' \leq x_t$*, comparing calendar dates.* [enforced] *by the token-state triggers; the check holds the six pairs exactly; the expiry condition since* `1.0.0-rc.37` *(before it, reporting a credential lost could promote a reserve already past its date). The date compared is the UTC date whatever timezone the session set, since* `1.0.0-rc.47`*; before it a client's timezone moved it. Pinned by* `check_token_core_triggers_are_bound`*.*

## 3 The ten constraints

**Property 2** (C1: the audit of record is append-only)**.** *For every audit table $R$ and times $\tau_1 \leq \tau_2$: $R_{\tau_1} \subseteq R_{\tau_2}$, and every row present at $\tau_1$ is unchanged at $\tau_2$.* [enforced] *by a* `BEFORE UPDATE OR DELETE` *trigger on each table, whose function raises* `insufficient_privilege`*, and by the grants that keep the purge carve-out out of the application role's reach. The grants reach every partition of the four partitioned audit tables since* `1.0.0-rc.40` *(before it the application role could empty them through their partitions), and the carve-out requires the purge's owner as well as its setting. Where a row is written only by a recorder (the change records, since* `1.0.0-rc.38`*; the lifecycle log, since* `1.0.0-rc.40`*), the application role cannot write one directly, so $R_{\tau_2} \setminus R_{\tau_1}$ holds only rows a recorder wrote. Pinned by* `check_aor_append_only_triggers`*,* `check_aor_privilege_boundary`*.*

**Property 3** (C2: a zero-knowledge verification stores no token identifier)**.** *For every event $e$ with disclosure level $d(e) \in \{\text{ZK}, \text{SELECTIVE}, \text{FULL}\}$: $d(e) = \text{ZK} \Rightarrow tok(e) = \bot$ and $d(e) = \text{FULL} \Rightarrow tok(e) \neq \bot$; SELECTIVE constrains neither.* [enforced] *by a check constraint;* [modelled]*. Pinned by* `check_c2_zk_token_null`*.*

**Property 4** (C3: one active token per person)**.** $\forall p \in \mathcal{P} : \left|\{t \in \mathcal{T} : p_t = p \wedge s_t = \text{ACTIVE}\}\right| \leq 1$. [enforced] *by a partial unique index on the person where the status is* ACTIVE*;* [tested] *with real concurrent threads;* [modelled]*. Pinned by* `check_one_active_token_index`*.* [limit]*: $p \in \mathcal{P}$ is an enrolment record, not a human being; one human enrolled as two records is two elements of $\mathcal{P}$ and nothing here sees it. C3 is credential uniqueness per enrolment identity, not human uniqueness, and not Sybil resistance.*

**Property 5** (C4: failed-login counting is atomic)**.** *If $k$ failed attempts against one account complete, in any interleaving, the counter increases by exactly $k$: no increment is lost.* [enforced] *by a single* `UPDATE ... RETURNING` *under the row lock. Pinned by* `check_c4_atomic_failed_login`*.*

**Property 6** (C5: no inline scripts)**.** *The set of script sources a page may execute is* {`'self'`}*.* [enforced] *by the response's Content-Security-Policy header. Pinned by* `check_csp_forbids_unsafe_inline`*.*

**Property 7** (C6: disclosure is decided by the server)**.** *Whatever a client submits, the stored event satisfies C2, and every page of the application except the admin and auditor SQL console returns no location for a* ZK *event: $d(e) = \text{ZK} \Rightarrow loc(\text{page}(e)) = \bot$. The row keeps the location the verifier states; that console, which runs the caller's own query, and the owner-role archive export return it as stored.* [enforced] *in the form handler, the C2 constraint, the Atlas functions and the queries the pages run. Pinned by* `check_c6_atlas_redacts_zk_location` *(the Atlas functions),* `check_c6_app_read_paths_redact` *(the page queries that name a location) and* `check_c2_zk_token_null` *(a* ZK *row names no token). That the investigation pages read views keyed by token or holder, and render no location, is not pinned by a check.*

**Property 8** (C7: cryptography is data)**.** *$\mathcal{G}$ is a relation (rows of a table), each token references a row of it, and algorithm metadata is obtained by joining that relation, never from a constant in application code. The accepted-signer allowlist is deliberately code, a security decision.* [enforced]*. Pinned by* `check_crypto_algorithm_is_data`*.*

**Property 9** (C8: every Atlas aggregate is bounded)**.** *For every Atlas query $q$ with a caller-supplied count $n$, the rows returned satisfy $|q| \leq \min(n, N_q)$ with a fixed $N_q$, and queries without a caller count return a fixed shape.* [enforced] *by route-level clamps passed into the SQL. Pinned by* `check_c8_atlas_caps`*.*

**Property 10** (C9: concurrency is tested with real threads)**.** *C3 and C4 hold in the interleavings produced by real concurrent threads against a live database.* [tested]*; this is a statement about a test suite, not a proof over all interleavings. Pinned by* `check_c9_concurrency_threading`*.*

**Property 11** (C10: identity is not money)**.** *No table and no column in the schema or its migrations has a monetary name or type.* [enforced] *structurally by the absence, and by a check that reads every name. Pinned by* `check_c10_no_money_tables`*.*

(An earlier draft wrote C10 as "no function $\mathcal{T} \to \mathbb{R}$". That is false: an expiry date is such a function. The constraint is about what the schema can *hold*, a monetary quantity, not about numbers.)

# 4 Authenticity and authorization

**Definition 3** (Signature)**.** $h_t = \text{SHA3-256}(\nu_t)$ *and* $\sigma_t = \text{Sign}_{\alpha_t}(k^-_{a_t}, h_t)$*. Authenticity is* $A(t) = \text{Ver}_{\alpha_t}(k^+_{a_t}, h_t, \sigma_t) \in \{0, 1\}$*, a function of the token and the published key alone, so it does not change with time.*

**Property 12** (Accepted algorithms)**.** $\alpha_t \in \mathcal{G}_{\text{acc}} = \{$*ML-DSA-65, ML-DSA-87*$\}$*; ML-DSA-44 is refused. A development placeholder signature exists and is never authentic offline.* [enforced].

**Property 13** (Two witnesses)**.** *At issuance, two independent ML-DSA implementations must both return 1. At verification, a verifier with implementations* $V_1, \ldots, V_m$ *available reports authentic iff* $m \geq 1$ *and all available ones return 1; if two disagree it reports not authentic.* [enforced].

*Limit.* Whether Module-LWE is hard is [assumed]. Polaris makes no claim of safety against quantum adversaries, and the classical half of a credential in migration is protected by nothing here.

**Property 14** (Authorization)**.** $R(t, \tau) = \mathbf{1}[s_t(\tau) = \text{ACTIVE} \wedge (x_t = \bot \vee \tau \leq x_t)]$*, where* $\tau \leq x_t$ *compares calendar dates and an expiry that cannot be read counts as passed. The relying-party answer is* $U(t, \tau) = A(t) \wedge R(t, \tau)$*, and* $A(t) = 1$ *implies nothing about* $R$*.* [enforced] *in both verification endpoints;* [tested].

# 5 Offline authorization: the status assertion

A status assertion is $\zeta = (\nu, s, \tau_i, \tau_e, \sigma)$ signed by the issuer. A verifier with clock $\tau$, maximum window $\Delta_v$ and a credential $t$ accepts it iff

$$\text{Ver}(\sigma) = 1 \ \wedge \ \nu = \nu_t \ \wedge \ k^+_\zeta = k^+_{a_t} \ \wedge \ \tau_i \leq \tau < \tau_e \ \wedge \ \tau_e - \tau_i \leq \Delta_v \ \wedge \ s = \text{ACTIVE}.$$

The window is half-open. [enforced]; each conjunct separately [tested] (2026-09-23). On the issuer's side the assertion states the same authorization $R(t, \tau)$ the online answer does: a credential past its expiry is asserted EXPIRED even while its stored status is ACTIVE, and an ACTIVE assertion's $\tau_e$ is no later than the end of the credential's last valid day. [enforced] since `1.0.0-rc.8`; before it the offline and online answers disagreed about expired credentials.

*Limit.* A credential revoked at $\tau_r < \tau_e$ still passes this check for $\tau \in [\tau_r, \tau_e)$. Offline revocation latency is therefore at most $\tau_e - \tau_i \leq \Delta_v$, and no offline mechanism can do better without a fresher assertion.

# 6 Privacy

**Property 15** (Issuer-unlinkable membership)**.** *In zero-knowledge mode the holder holds a secret $w$. The leaf for context $c$ is $\ell = \text{Poseidon}(w \,\|\, c)$; an epoch publishes the Merkle root $\rho$ over its leaves. A proof $\pi$ shows knowledge of $(w, path)$ with $\ell$ in the tree of root $\rho$, bound to the public inputs $(\rho, epoch, c, nonce, scope, \lambda)$, and reveals neither $w$ nor the leaf's index. The nullifier is $\lambda = \text{Poseidon}(w \,\|\, scope \,\|\, epoch)$.* [enforced] *by the circuit; recomputed by an independent second witness;* [assumed]*: the proof system's soundness and zero knowledge, and Poseidon behaving as a pseudorandom function.*

**Property 16** (What a verifier learns)**.** *Let $N$ be the number of leaves in the epoch's tree for that context (the anonymity set). A zero-knowledge presentation identifies the holder's leaf to the verifier with probability no better than $1/N$ plus the assumptions' advantage. In particular $N = 1$ identifies the holder. Within one scope and epoch, two presentations by one holder share $\lambda$ (one enrolment record, one proof); across scopes or epochs the nullifiers are unlinkable under the assumption above.* [assumed] *bound. Since* `1.0.0-rc.46` *an epoch closes only if $N \geq N_{\min}$ (a database setting,* 20 *unless set) and a proof against an epoch with $N < N_{\min}$ is answered* privacy unavailable*, never accepted;* [enforced] *at close, at verification, and offline by the detached verifier from the checkpoint's signed $N$. Before it nothing enforced a minimum $N$.* [limit]*: $N_{\min}$ bounds the set, not what epoch timing reveals.*

(An earlier draft bounded identification by $1/|\mathcal{P}|$. The anonymity set is the epoch's leaf set, not the population.)

**Property 17** (Pairwise identifiers)**.** *A relying party that keys its records on the handle $\psi_r$ (from the holder's bound key), or receives the login subject $sub_r$, where*

$$\psi_r = \text{SHA3-256}(\texttt{polaris-pairwise/1} \,|\, k^{+}_{\text{holder}} \,|\, r),$$
$$sub_r = \text{SHA3-256}(\texttt{polaris-pairwise/1} \,|\, \nu \,|\, client_r),$$

*stores values that differ for $r \neq r'$.* [enforced].

*Limit.* A full presentation still *shows* each verifier the same $(\nu, \sigma, k^{+})$; two verifiers that keep that material link on it with certainty. Correlation is bounded, not eliminated: what the design bounds is what a verifier needs to store.

# 7 Duress

**Property 18** (Duress-aware presentation)**.** *A card holds two key slots, normal and duress, selected by two PINs of equal length, with both comparisons always run and the retry counter treated the same. The reader at the counter receives a well-formed response under either PIN and reports the same outcome; only the authority, which recorded both slots' public keys at personalization, decides which slot signed, and that is where an alert can be raised. A holder proof is signed without the presented code, so a coerced presentation is byte-indistinguishable from a consenting one.* [enforced] *in the card profile and emulator;* [tested].

*Limit.* The mechanism resists a coercer who does not know it exists. It does not resist one watching the holder, and against lawful or institutional access it is net-negative, because the duress record is append-only (C1). It is not a claim of resistance to compulsion.

## 8 Federation

**Definition 4** (Trust edges)**.** *$\mathcal{F} = (\mathcal{A}, E)$ with $(a_i, a_j, c, [\tau_0, \tau_1)) \in E$ when a fresh manifest signed by an active anchor of $a_i$ attests $a_j$'s key for context $c$ within that window. Write $T(a_i, a_j, c, \tau)$.*

**Property 19** (No transitivity)**.** *$T(a_i, a_j, c, \tau) \wedge T(a_j, a_k, c, \tau) \not\Rightarrow T(a_i, a_k, c, \tau)$: acceptance consults only edges from authorities the relying party itself trusts. A foreign credential is accepted iff $A(t) \wedge \exists a : T(a, a_t, c, \tau) \wedge \neg\mathrm{Rev}(t, \tau)$, where revocation comes from the issuer's own authentic, fresh feed. No central service sits in the path.* [enforced]*; each conjunct* [tested] *(2026-09-23).*

## 9 Transparency

**Property 20** (Append-only log)**.** *Log states $L_n = (x_1, \ldots, x_n)$; $L_m \prec L_n$ iff $m \leq n$ and they agree on the first $m$ entries. A signed head is $\theta_n = (n, r_n, \tau_n, \sigma_\ell)$ with $r_n$ the RFC 6962 Merkle root. For $m < n$ a consistency proof shows $L_m \prec L_n$; an inclusion proof shows $x_i \in L_n$.* [enforced]*.*

**Property 21** (Equivocation is evidence)**.** *Two authentic heads $\theta_n, \theta'_n$ of the same size with $r_n \neq r'_n$ are a proof that the log operator equivocated. A head is* witnessed *when at least $K$ distinct trusted witnesses have cosigned $(n, r_n)$.* [enforced]*.*

## 10 Delegated agents

**Property 22** (Agent grants)**.** *A grant $\kappa$ signed by the holder names an agent key, actions $\mathcal{U} \neq \varnothing$, limits and a window. A request for action $u$ with amount $m$ is usable iff the grant is authentic and fresh, the holder key is bound by an issuer to this credential and that binding is fresh, $u \in \mathcal{U}$, the limits hold (an unknown limit refuses), the holder has not revoked the grant, and the agent signed this service's nonce for this action. A grant altered after signing is not authentic; revoking a grant does not revoke the credential.* [enforced]*; each conjunct* [tested]*.*

## 11 Presentations over constrained channels

**Property 23** (QR bounds)**.** *A QR presentation has at most* 9999 *frames, at most* 512 *KiB of compressed payload and at most* 2 *MiB after decompression, and it verifies only if the digest over the reassembled frames matches the one the frames carry.* [enforced]*.*

## 12 Algorithm migration

**Property 24** (No credential verifies under nothing)**.** *Let $\mathcal{G}_{\mathrm{acc}}(\tau)$ be the algorithms accepted at time $\tau$. During a migration every active credential keeps at least one signature under an accepted algorithm:*

$$\forall\tau\ \forall t:\ s_t(\tau) = \textsc{active}\ \Rightarrow\ \exists\sigma \in \Sigma_t:\ \alpha(\sigma) \in \mathcal{G}_{\mathrm{acc}}(\tau) \wedge \mathrm{Ver}(\sigma) = 1.$$

*The superseded signature keeps verifying until its algorithm's deprecation date, and deprecating an algorithm is refused while any active credential depends on it alone.* [enforced]*; the migration drill samples the number of credentials violating the property at every batch boundary, not only at the ends, and measures it at zero* [measured]*.*

*Limit.* Agility is issuer-side only. A deprecation date reaches no signed artifact, so a verifier cannot learn that an algorithm is deprecated: retiring one is a software release to every relying party. The classical half of a credential in a hybrid cutover is protected by nothing here. Migrating back onto an algorithm the population left cannot re-sign a credential already holding a deprecated signature under it, since a credential holds at most one signature per algorithm and none changes, $|\{\sigma \in \Sigma_t : \alpha(\sigma) = a\}| \leq 1$. Such credentials are counted as blocked, stay on the signature they have, and hold the window open until they are re-issued [enforced] since `1.0.0-rc.9`; before it the run stopped without saying so.

## 13 Exchanges and timestamps

**Property 25** (A receipt records that an exchange happened, not what was exchanged)**.** *For an exchange of request $q$ and response $r$ between authorities $a_i$ and $a_j$ in context $c$, the signed receipt is $J = (a_i, a_j, c, H(q), H(r), via, \tau)$; neither $q$ nor $r$ is in $J$.* [enforced] *by the receipt's signed field set.*

**Property 26** (Timestamps)**.** *A timestamp is a signature over $(H(x), nonce, \tau)$; it shows that the digest existed at $\tau$. Anchored in the transparency log, a timestamp forged later must be absent from every witnessed head of its claimed era.* [enforced]*.*

*Limit.* A digest is not a secret. For a document an adversary can guess, $H(x)$ confirms the guess; a timestamp hides only what is hard to guess. An earlier draft wrote $I(x; \vartheta) \approx 0$, which is false in exactly that case.

## 14 Key custody

**Property 27** (The application does not hold the signing key)**.** *The issuer's signing key $k^-$ is held by a custody driver behind one interface: a PKCS#11 token, where the key is non-extractable and signing happens inside the token, a managed key service, or, for development, a software key. The application requests signatures and never reads $k^-$ in the first two; secrets for the driver are read from files, and the application refuses to start if they are passed in the environment.* [enforced]*.*

## 15 Recovery

**Property 28** (Recovery point and recovery time)**.** *For a backup taken continuously and restored after a loss at $\tau$, the restored database equals the database at some $\tau - \delta$, with targets $\delta \leq 300\,s$ (RPO) and restoration within $14\,400\,s$ (RTO). Two drills at `v9.192` measured $\delta$ of 36 and 42 seconds and a database restore under three seconds, on notional data.* [measured] *at that version; later versions are not re-measured in this document.*

## 16 Operations and scale

**Property 29** (Re-signing throughput)**.** *Re-signing under ML-DSA-87 with one custodied key runs at about 345 credentials per second on one runner, 91 percent of it signing.* [measured]*. A population of $3.5 \times 10^8$ is then about $1.0 \times 10^6$ s (twelve days) on one runner; on 64 runners,* if the rate scales linearly*, about $1.6 \times 10^4$ s. The second figure is an extrapolation, not a measurement.*

**Property 30** (Every id space outlasts the horizon)**.** *For every sequence $q$ in the schema, with capacity $c_q \in \{2^{31}-1,\ 2^{63}-1\}$ and a growth rate $r_q$ at the roadmap's targets, $c_q / r_q \geq H$ with $H = 25$*

*years; a sequence whose table does not grow with the population is declared bounded instead, with its reason.* [enforced] *by* `check_capacity_model` *over every sequence since* `1.0.0-rc.14`*. Before it the check sized nine of forty, and three 32-bit sequences on the enrollment path had* $c_q/r_q$ *of* 9.8, 9.8 *and* 14.7 *years. The rates are assumptions stated beside each figure, except the per-verification and per-epoch ones.*

## 17 Analytics

**Property 31** (The Atlas)**.** *Atlas outputs are aggregates over events, bounded by C8* [enforced]*, with the locations of* ZK *events withheld by C6* [enforced]*. That no output carries a per-person row is the design; no single check pins it.*

*Limit.* Aggregates are information about people. The Atlas is bounded, not information-free, and an earlier draft's claim that its output has zero mutual information with the population is false.

## 18 The assurance layer

For each documented guarantee $g$ with a check $I_g$ and a detection test $D_g$: $D_g$ asserts $I_g$ passes on a good fixture and fails on a broken one. That establishes *fixture* sensitivity. Mechanism sensitivity is the stronger property

$$\forall \mu \in M_g : \quad I_g(\mu(\mathfrak{P})) = 0,$$

for a set $M_g$ of mutations that break $g$'s mechanism in the real tree. On 2026-09-23 held-out rounds found many $\mu$ with $I_g(\mu(\mathfrak{P})) = 1$, and the repairs add tests until the listed mutations are caught. A passing test implies nothing beyond the inputs it exercises: $T_g = 1 \not\Rightarrow g$. [measured]; the rounds and their counts are recorded in the report.

## 19 The operating contract

A change $\Delta\mathfrak{P}$ to product behaviour is admitted iff it has exactly one qualifying reason:

$$\Delta\mathfrak{P} \text{ admitted} \iff E_{\text{interop}} \vee E_{\text{user}} \vee E_{\text{security}} \vee \exists z : \ \text{Promise}(z) = 1 \wedge \text{Impl}(z) = 0,$$

the last being an executable counterexample to a published promise. Progress is the set of external dependencies $\mathcal{E}_{\text{ext}}$. On 10 October 2026 it holds fifteen third-party implementations whose presentations the published verifier accepted (six of them wallets run unmodified), one hosted conformance plan, certified on 24 September 2026, and one third-party test corpus. $|\mathcal{E}_{\text{int}}| \to \infty \not\Rightarrow |\mathcal{E}_{\text{ext}}| > 0$.

## What this document does not do

It does not prove the properties; the marks say what supports each one. It omits a composite "integrity score": an earlier draft wrote Polaris as the maximiser of a product $A \cdot R \cdot Z \cdot S$, but no such objective is defined, measured or optimised anywhere in the repository, and a formula that looks like a result would be read as one.

